



Chapter 3 — System Architecture

3.1 Architecture Philosophy

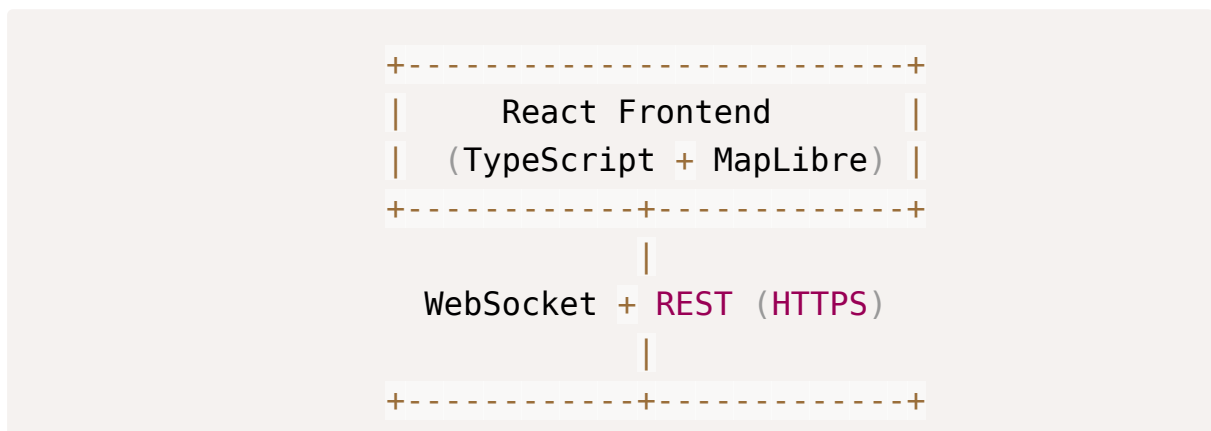
DOIP's architecture follows a “**monolith-first, microservices-by-evolution**” strategy:

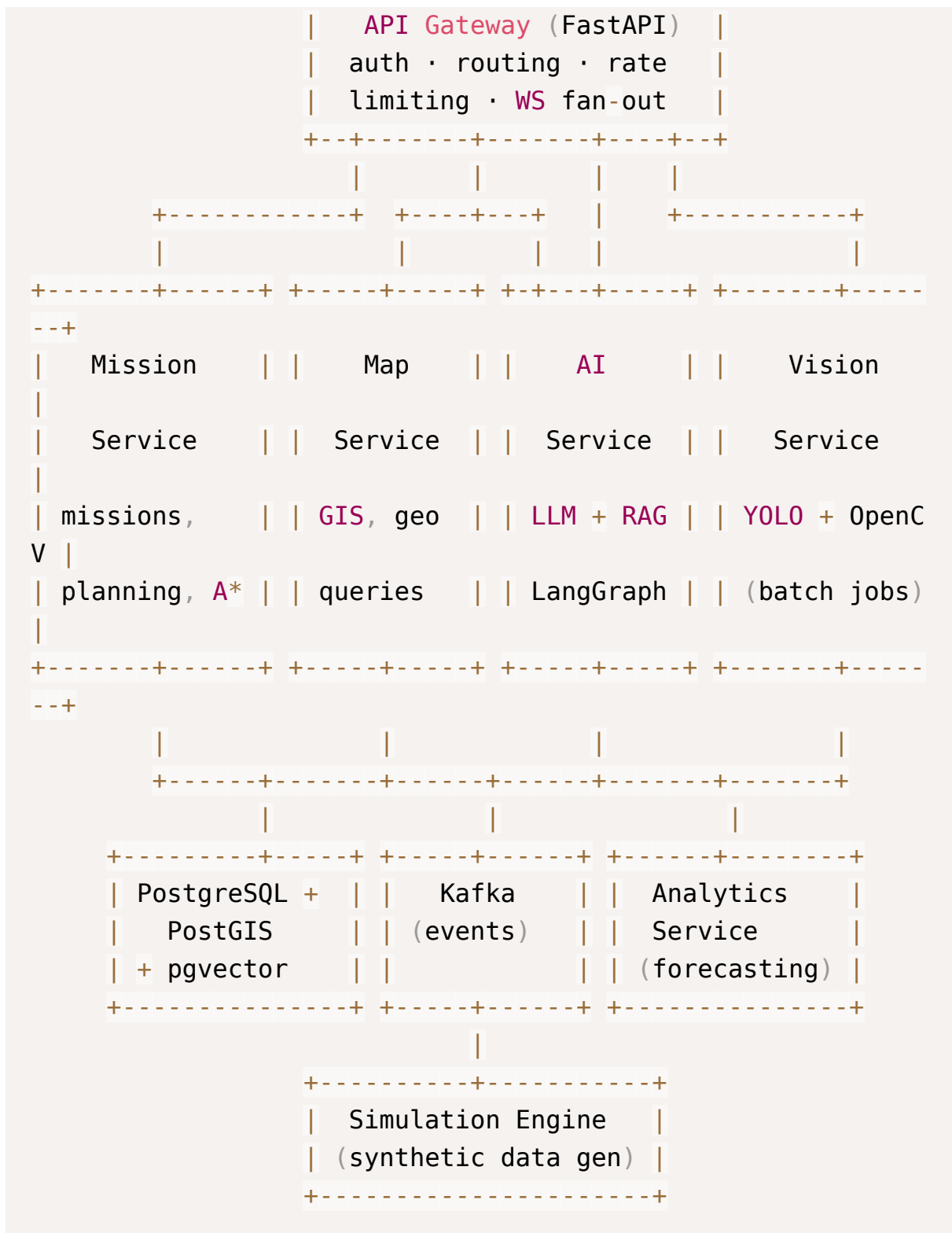
1. **Phases 1–3:** A single **modular monolith** (FastAPI) with clean internal package boundaries, plus a separate AI service. Real-time transport via Redis Pub/Sub → WebSocket.
2. **Phases 4–6:** Heavy workloads (computer vision, analytics) split into their own services because they have genuinely different resource profiles (GPU/CPU-bound, batch-oriented).
3. **Phase 7:** Kafka replaces Redis Pub/Sub as the event backbone; Kubernetes replaces Docker Compose. Both are executed as **documented migrations** — which is itself a valuable engineering artifact.



Splitting services *when the need appears* — and being able to explain why — demonstrates stronger architectural judgment than starting with 7 microservices on day one.

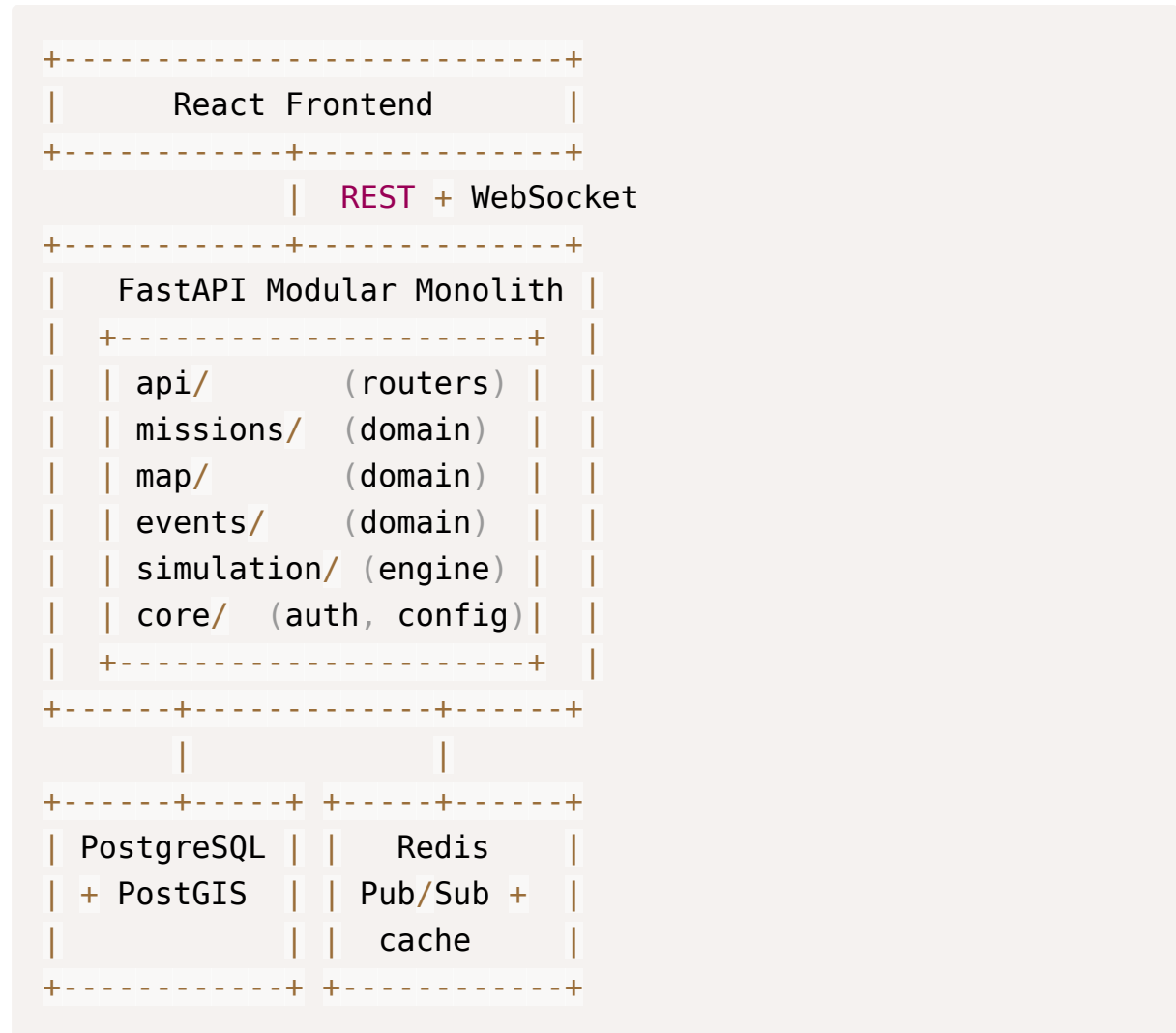
3.2 Target Architecture (Final State, Phase 7)





Two extension components (added in v1.3) attach to this backbone without changing it: the War-Room Session Manager (Ch. 15) and the Red-Team Agent (Ch. 16) — see §3.10 for how they plug in.

3.3 Phase-1 Architecture (What You Actually Build First)



Same external contract (REST + WebSocket) as the final architecture — the frontend never has to change when the backend decomposes.

3.4 Component Responsibilities

Component	Responsibilities	Key technologies
Frontend	Dashboard, live map, planning UI, chat panel, admin screens	React, TypeScript, MapLibre GL, Zustand, TanStack Query
API Gateway	AuthN/AuthZ, request routing, rate limiting, WebSocket connection management & fan-out	FastAPI, Uvicorn

Component	Responsibilities	Key technologies
Mission Service	Mission CRUD, waypoints, A*/Dijkstra route optimization, feasibility & battery model	Python, NetworkX/custom grid
Map Service	Spatial queries, sectors, zones, GeoJSON APIs, movement history/replay	PostGIS, GeoAlchemy2, Shapely
AI Service	Situation reports, operator chat, mission briefs, RAG pipeline	Ollama (Llama 3 / Qwen), LangGraph, pgvector
Vision Service	Frame extraction, YOLO inference, risk scoring, detection events	YOLOv8/v11 (Ultralytics), OpenCV, Celery workers
Analytics Service	Forecast training/serving, after-action reports, model registry	XGBoost, Prophet, scikit-learn, (optional LSTM/PyTorch)
Simulation Engine	Tick loop, scenario execution, entity movement, synthetic sensors, weather, health, consumption	Python asyncio (Chapter 7)
Data layer	System of record + vector search + geospatial	PostgreSQL 16, PostGIS, pgvector
Event backbone	Event distribution between components	Redis Pub/Sub → Kafka (Phase 7)
War-Room Session Manager (Ch. 15)	Multi-operator sessions: membership & session roles, sector assignments, annotations, task board, presence coordination	FastAPI module (in monolith), PostGIS geometries, evented state
Red-Team Agent (Ch. 16)	Adaptive training adversary: performance assessment, intensity control, inject planning — all through the Validation Gate	Python (rule policy v1, optional LLM v2), Inject Playbook, scenario DSL

3.5 Data Flow — The Life of an Event

1. **Generation:** The simulation engine ticks and produces a typed event, e.g. `IncidentCreated`.
2. **Persistence:** The event is written to the `events` table (append-only) and domain tables are updated (`incidents` row created).
3. **Publication:** The event is published to the event backbone on topic `events.incident`.

4. **Fan-out:** The gateway's WebSocket manager receives it and pushes to all subscribed clients (filtered by run + layer subscriptions).
5. **Render:** The frontend reducer routes the event to the map layer, event feed, and incident widget; all update in < 1 s.
6. **AI consumption:** The AI service buffers events; on the next report cycle they become part of the situation summary context.
7. **Analytics consumption:** The analytics service appends telemetry for future model training.
8. **Replay:** Because events are append-only with sim-timestamps, any run can be replayed deterministically from the DB.

3.6 Communication Patterns

Pattern	Used for	Notes
REST (JSON)	CRUD, queries, commands (create mission, start scenario)	OpenAPI auto-docs from FastAPI
WebSocket	Server → client live updates	One connection per client; topic subscription protocol (Chapter 6)
Pub/Sub (Redis → Kafka)	Service → service events	At-least-once; consumers are idempotent
Async jobs (Celery/ARQ)	Video processing, model training, report generation	Progress events published back onto the bus
SSE (fallback)	AI token streaming to chat panel	Simpler than WS for one-way LLM streams

3.7 Key Architectural Decisions (ADR Summary)

#	Decision	Rationale
ADR-1	Modular monolith first	One deployable, fast iteration; boundaries enforced by package structure and import-linting
ADR-2	Redis Pub/Sub before Kafka	Identical demo behavior at 5% of ops cost; Kafka arrives when multi-service fan-out and replay justify it

#	Decision	Rationale
ADR-3	PostgreSQL as single source of truth	PostGIS (geo) + pgvector (RAG) + JSONB (events) in one engine — no premature polyglot persistence
ADR-4	Separate AI service from day one	Different dependency stack (GPU/Ollama), different release cadence, isolates failures
ADR-5	Event-sourcing-lite (append-only events + mutable projections)	Enables replay and after-action review without full CQRS complexity
ADR-6	Simulation as the sole writer of operational state	One authoritative producer eliminates data races; UI issues <i>commands</i> , engine emits <i>events</i>
ADR-7	GeoJSON everywhere on the wire	Native to MapLibre and PostGIS; zero client-side conversion
ADR-8	Local LLMs via Ollama with pluggable provider interface	Free, private, offline-capable; can swap to hosted APIs via config

3.8 Scalability & Resilience Notes

- **WebSocket fan-out** is the first bottleneck at scale; mitigated by per-layer subscriptions and server-side batching of position updates (e.g., 4 Hz frames instead of per-event pushes).
- **Vision and analytics** run as queue-fed batch workers — they can lag without affecting the live picture.
- **Backpressure**: if a client falls behind, the gateway drops intermediate position frames but never drops incident/alert events.
- **Recovery**: on restart, clients re-fetch a state snapshot via REST, then resume the WS stream from the last event ID (gap-free resume).
- **Idempotency**: all event consumers key on `event_id`; duplicate delivery is harmless.

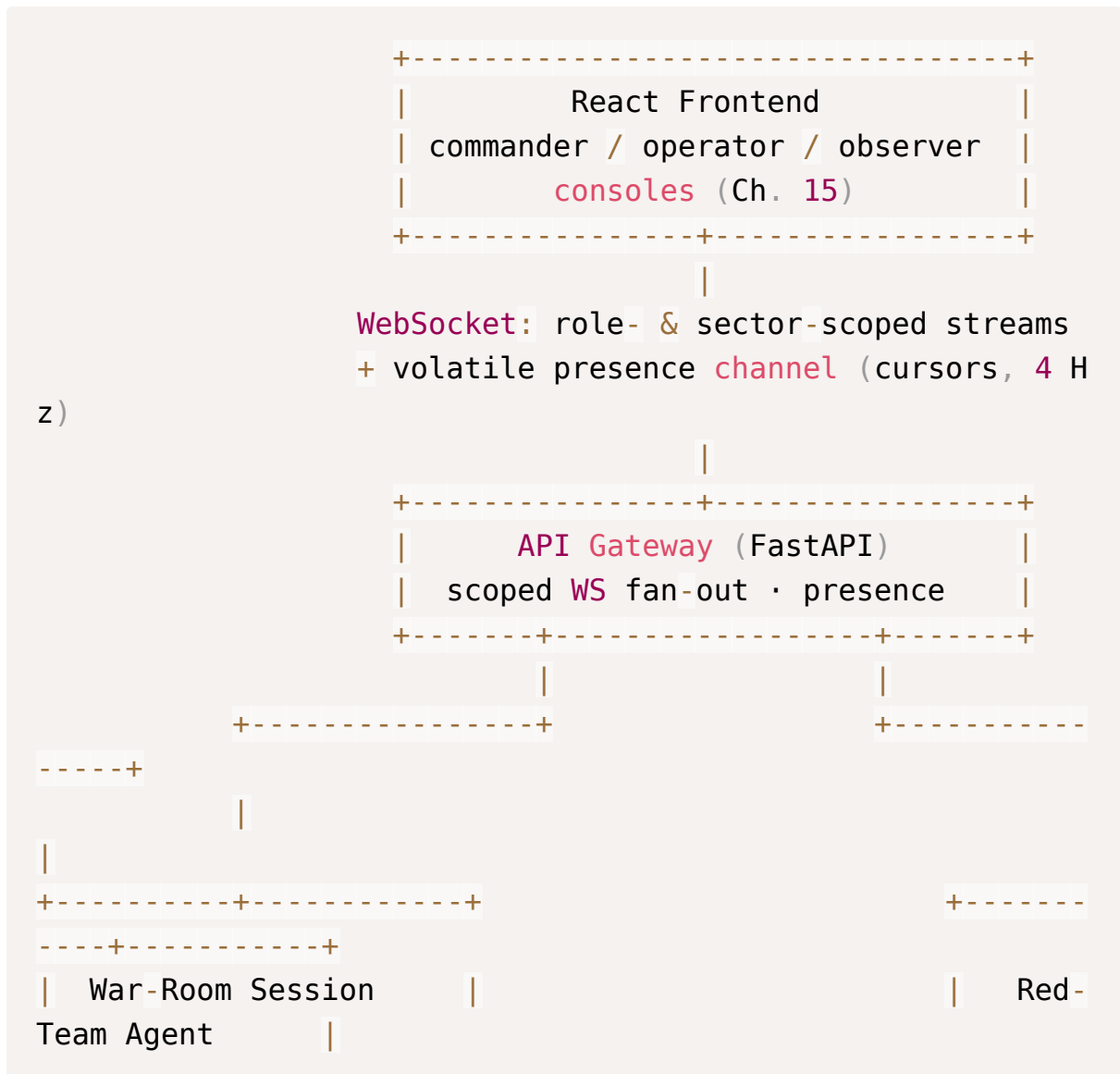
3.9 Migration Milestones

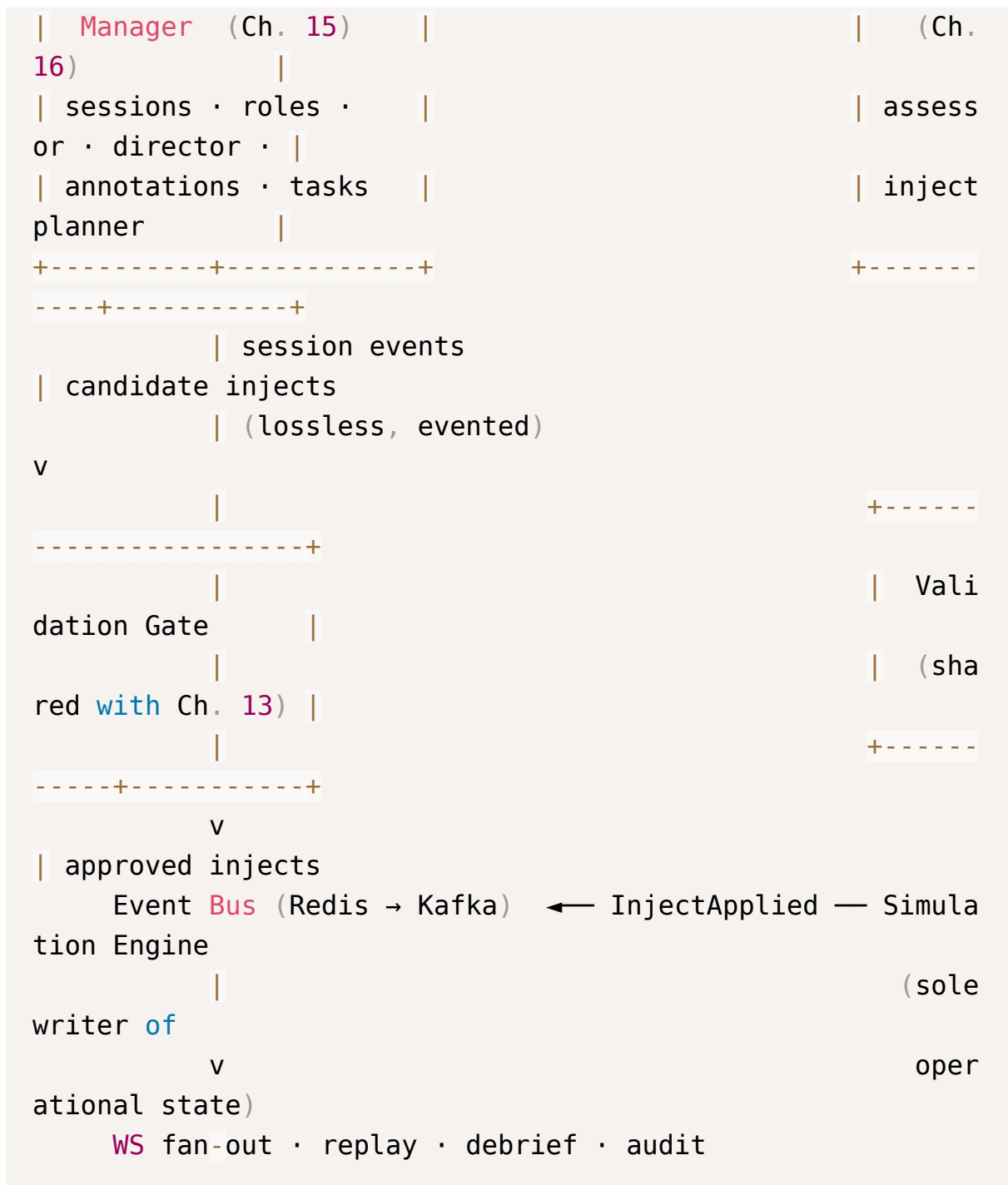
Milestone	Trigger	Work
Split Vision Service	Phase 4 start	Extract to container with GPU access, Celery queue, artifact storage

Milestone	Trigger	Work
Split Analytics Service	Phase 6 start	Extract training/serving, add model registry
Redis → Kafka	Phase 7	Topic design, consumer groups, replay tooling, dual-write cutover
Compose → Kubernetes	Phase 7	Manifests/Helm, HPA for vision workers, Ingress + TLS

3.10 Extension Components — Collaboration & Adversarial Training (Ch. 15–16)

Both extensions plug into the existing event backbone as **peers of the simulation engine's consumers** — neither modifies the engine or the core data flow (ADR-6 intact):





Integration rules that keep the core architecture clean:

1. **Session state is evented.** Annotations, tasks, and membership changes are standard events — they inherit persistence, resume, replay, and audit for free (no parallel state system).
2. **Presence is volatile.** Cursor/awareness frames use a lossy side-channel like position frames; they never touch the event log.

3. **The RTA is a runtime scenario author, not a second engine.** It emits only whitelisted DSL actions through the same Validation Gate as the AI scenario author (Ch. 13); the engine remains the sole writer of operational state.
4. **Determinism survives both extensions.** RTA decisions are recorded as `InjectApplied` events (replayed tick-exactly); session events carry sim-timestamps and appear in debrief replay at their original moments.
5. **Scoped fan-out strengthens, not weakens, the gateway.** Sector/role filtering is server-side and reduces per-client traffic — the first fan-out bottleneck (§3.8) gets *better* with War-Room Mode.